Guia: Azure, AKS, Terraform, SQL Server e Kafka para Produção

Oct 2, 2026 · @Janderson

Como usar este guia e o cenário-alvo

1 milhão de requests por dia entre 8h e 20h dá cerca de 23 requests por segundo (RPS) em média e algo entre 70 e 120 RPS no pico. É uma carga moderada: uma stack bem configurada com 3 a 6 pods e um banco de tier médio atende com folga. O maior risco não é escala, é excesso de complexidade.

$$\text{RPS médio} = \frac{1.000.000}{12 \times 3600} \approx 23 \qquad \text{RPS pico} \approx 23 \times (3 \text{ a } 5) \approx 70\text{–}120$$

Regra de projeto: dimensione para 2x o pico estimado (cerca de 250 RPS) e teste com carga real antes de ir para produção. Isso cobre picos inesperados, como campanhas de marketing ou retries em cascata.

Como estudar: leia as seções 1 a 5 na ordem (cada tecnologia depende da anterior), depois a arquitetura de referência (6) e o checklist (7). Siga a trilha (8), fazendo os laboratórios (9) à medida que avança. Use as perguntas (10) para se testar ao fim de cada semana.

Como as peças se encaixam:

Peça	Papel na stack	Analogia
Azure	A nuvem: onde tudo roda	O terreno e a infraestrutura da cidade
Terraform	Cria e versiona a infraestrutura como código	A planta do arquiteto
AKS (Kubernetes)	Roda e escala sua aplicação em containers	O condomínio que aloca apartamentos
SQL Server	Guarda os dados transacionais	O cofre com registro de tudo
Kafka	Transporta eventos entre serviços de forma assíncrona	O correio que nunca perde carta

1. Fundamentos de cloud e Azure

Azure é a nuvem da Microsoft: você aluga computação, rede e armazenamento sob demanda e paga pelo uso. Para produção, os conceitos que mais importam são hierarquia de recursos, identidade, rede e custo.

Modelos de serviço

Modelo	Você gerencia	Azure gerencia	Exemplo
IaaS	SO, runtime, app, dados	Hardware, rede física	Virtual Machines
PaaS	App e dados	SO, patches, runtime	Azure SQL Database, App Service
CaaS	Containers e manifestos	Control plane do Kubernetes	AKS
SaaS	Só o uso	Tudo	Microsoft 365

Na prática, “IaaS com Terraform” quer dizer descrever toda a infraestrutura (inclusive os serviços PaaS) em código. Prefira PaaS sempre que puder: menos patch, menos plantão.

Hierarquia de recursos

1. **Tenant (Microsoft Entra ID):** a organização e seus usuários.
2. **Management Groups:** agrupam subscriptions para aplicar políticas.
3. **Subscription:** unidade de cobrança e limite. Use uma para produção e outra para dev/homologação.
4. **Resource Group:** pasta lógica de recursos com o mesmo ciclo de vida (ex.: `rg-app-prd-brazilsouth`).
5. **Resource:** a VM, o banco, o cluster.

Regiões e zonas de disponibilidade

- **Região:** um conjunto de datacenters (ex.: Brazil South, em São Paulo). Escolha a mais próxima dos usuários por latência e LGPD.
- **Availability Zone (AZ):** datacenters fisicamente separados dentro da região. Distribua nós e banco em 3 zonas para sobreviver à queda de um datacenter.
- **Par de regiões:** Brazil South pareia com South Central US. Relevante para disaster recovery; confira preço e disponibilidade de serviços, que variam por região.

Identidade e acesso

- **Microsoft Entra ID** (antigo Azure AD): login de pessoas e aplicações.
- **RBAC:** papéis (Reader, Contributor, Owner ou customizados) atribuídos num escopo. Princípio do menor privilégio: ninguém é Owner em produção no dia a dia.
- **Managed Identity:** identidade da própria aplicação, sem senha no código. No AKS, use **Workload Identity** para que o pod acesse Key Vault ou SQL sem segredo.
- **Key Vault:** cofre para segredos, chaves e certificados.

Rede

- **VNet:** sua rede privada (ex.: `10.10.0.0/16`), dividida em **subnets** (AKS, banco, gateway).
- **NSG:** firewall de subnet ou placa de rede, com regras de porta e origem.
- **Private Endpoint:** coloca um serviço PaaS (SQL, Key Vault) dentro da sua VNet com IP privado. Em produção, o banco não deve ter acesso público.
- **Application Gateway com WAF** ou **Azure Front Door:** entrada HTTP pública com proteção contra ataques (OWASP).
- **Private DNS Zones:** resolvem os nomes dos private endpoints.

Custos

- Pague pelo uso, mas os maiores gastos costumam ser nós do AKS, o banco e o tráfego de saída.
- **Reservations / Savings Plans** (1 ou 3 anos) reduzem bastante o custo de carga estável.
- Configure **Budgets** e alertas no Cost Management desde o primeiro dia.
- **Tags** obrigatórias (`env`, `owner`, `cost-center`) permitem saber quem gasta o quê.
- Use a Calculadora de Preços do Azure antes de aprovar qualquer arquitetura.

Ferramentas do dia a dia

- **Portal Azure:** para explorar e aprender; não crie produção por cliques.
- **Azure CLI (`az`):** `az login`, `az group list`, `az aks get-credentials`.
- **Azure Monitor / Log Analytics / Application Insights:** métricas, logs e tracing.

2. Terraform e Infraestrutura como Código

Terraform descreve a infraestrutura em arquivos `.tf` declarativos: você diz *o que* quer, e ele calcula *como* chegar lá. Tudo fica versionado no Git, revisado em pull request e reproduzível entre ambientes.

Conceitos essenciais

Conceito	O que é
Provider	Plugin que conversa com uma API (`azurerm` , `azuread` , `kubernetes` , `helm`)
Resource	Um objeto criado e gerenciado (ex.: `azurerm_kubernetes_cluster`)
Data source	Leitura de algo que já existe, sem gerenciá-lo
Variable / Output	Entradas e saídas de um módulo
Module	Pasta reutilizável de recursos (ex.: módulo `network` , módulo `aks`)
State	Arquivo que mapeia o código para os recursos reais. É a peça mais crítica
Backend	Onde o state fica guardado (em Azure: um Storage Account com lock)

Ciclo de trabalho

1. `terraform init` : baixa providers e configura o backend.
2. `terraform fmt` e `terraform validate` : formatação e checagem de sintaxe.
3. `terraform plan -out=tfplan` : mostra o que vai mudar. **Leia sempre**, principalmente linhas com `destroy` ou `forces replacement` .
4. `terraform apply tfplan` : aplica exatamente o plano revisado.
5. Em produção, só o pipeline de CI/CD roda `apply` , nunca a máquina de alguém.

Regras de ouro para produção

- **State remoto e protegido:** Storage Account com versionamento, soft delete e acesso restrito. Nunca no Git: ele contém segredos.
- **Um state por ambiente** (dev, hml, prd), e idealmente por camada (rede, plataforma, aplicação), para limitar o estrago de um erro.
- **Fixe versões** do Terraform e dos providers (`required_providers`), e commite o `.terraform.lock.hcl` .
- **Nada manual no portal.** Mudança manual vira *drift*; rode `plan` agendado para detectar.
- **`prevent_destroy`** em recursos críticos (banco, Key Vault).

- Segredos vêm do Key Vault ou de variáveis do pipeline, nunca hardcoded.
- Ferramentas de apoio: `tflint`, `checkov` ou `trivy` para segurança, e os Azure Verified Modules como base.

Estrutura de repositório sugerida

```
infra/
  modules/
    network/      # VNet, subnets, NSGs
    aks/          # cluster, node pools
    sql/          # Azure SQL + private endpoint
    eventhubs/    # Kafka gerenciado
  envs/
    dev/  main.tf  backend.tf  terraform.tfvars
    prd/  main.tf  backend.tf  terraform.tfvars
```

Exemplo mínimo da stack

```
terraform {
  required_version = ">= 1.9"
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
  }
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstateprd"
    container_name       = "tfstate"
    key                  = "prd/platform.tfstate"
  }
}

provider "azurerm" { features {} }

locals {
  tags = { env = "prd", owner = "time-plataforma", cost-center = "app" }
}

resource "azurerm_resource_group" "rg" {
  name     = "rg-app-prd-brazilsouth"
  location = "brazilsouth"
  tags     = local.tags
}

resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-app-prd"
```

```
  address_space       = ["10.10.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "aks" {
  name                 = "snet-aks"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.10.0.0/20"]
}

resource "azurerm_kubernetes_cluster" "aks" {
  name                = "aks-app-prd"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  dns_prefix          = "aksappprd"
  sku_tier            = "Standard"   # SLA do control plane

  default_node_pool {
    name                = "system"
    vm_size             = "Standard_D4s_v5"
    vnet_subnet_id      = azurerm_subnet.aks.id
    zones               = ["1", "2", "3"]
    auto_scaling_enabled = true
    min_count           = 3
    max_count           = 6
  }

  identity { type = "SystemAssigned" }

  network_profile {
    network_plugin      = "azure"
    network_plugin_mode = "overlay"
  }

  oidc_issuer_enabled       = true
  workload_identity_enabled = true
  tags                      = local.tags
}
```

Confira nomes de argumentos na documentação do provider azurerm: eles mudam entre versões principais (ex.: a 4.x renomeou campos de autoscaling).

3. Kubernetes e AKS

Kubernetes orquestra containers: você declara "quero 4 cópias desta imagem, com 500m de CPU cada", e ele mantém isso vivo, reinicia o que cai e distribui a carga. O AKS é o Kubernetes gerenciado do Azure: a Microsoft cuida do control plane, e você cuida dos nós e das cargas.

Antes: Docker em 5 linhas

- **Imagem:** pacote imutável com app + dependências, construído por um `Dockerfile`.
- **Container:** uma imagem em execução.
- **Registry:** onde as imagens ficam. No Azure, o **Azure Container Registry (ACR)**.
- Use imagens pequenas (ex.: `distroless`, `alpine`), sem rodar como root, com tag imutável (versão ou hash do commit, nunca `latest`).

Objetos do Kubernetes que você vai usar

Objeto	Para que serve
Pod	Menor unidade: um ou mais containers juntos
Deployment	Mantém N réplicas de um pod e faz rolling update
Service	IP e DNS estáveis para um grupo de pods (ClusterIP, LoadBalancer)
Ingress / Gateway API	Roteia HTTP de fora para os Services (por host e path)
ConfigMap / Secret	Configuração e segredos injetados no pod
HorizontalPodAutoscaler (HPA)	Aumenta ou reduz réplicas por CPU, memória ou métrica customizada
PodDisruptionBudget (PDB)	Garante mínimo de pods vivos durante manutenção
Namespace	Separação lógica (por time ou por app)
Job / CronJob	Tarefas pontuais ou agendadas

Conceitos do AKS

- **Node pools:** grupos de VMs. Separe um pool *system* (componentes do cluster) de um pool *user* (sua aplicação).

- **Cluster Autoscaler:** adiciona ou remove nós quando faltam ou sobram recursos. Combine com o HPA: o HPA cria pods, e o autoscaler cria nós para eles.
- **KEDA:** autoscaling por eventos, como o lag de um tópico Kafka. É ideal para consumers.
- **Rede:** Azure CNI Overlay é o padrão recomendado (economiza IPs da VNet).
- **Ingress:** o add-on *application routing* (NGINX gerenciado) ou Application Gateway for Containers.
- **Upgrades:** o Kubernetes lança versões frequentes, e o AKS suporta apenas as mais recentes. Ative canal de auto-upgrade e janela de manutenção.
- **Tier Standard:** dá SLA ao control plane. Use em produção.

Exemplo de Deployment pronto para produção

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector: { matchLabels: { app: api } }
  template:
    metadata: { labels: { app: api } }
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector: { matchLabels: { app: api } }
      containers:
        - name: api
          image: acrappprd.azurecr.io/api:1.4.2
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "500m", memory: "512Mi" }
            limits:   { memory: "512Mi" }
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            initialDelaySeconds: 20
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: api }
```

```
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: api }
  minReplicas: 3
  maxReplicas: 12
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization:
65 } }
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: api }
spec:
  minAvailable: 2
  selector: { matchLabels: { app: api } }
```

O que cada parte garante

- **requests:** o scheduler reserva esse recurso; sem ele o HPA não funciona direito.
- **limit de memória:** evita que um pod derrube o nó. Limite de CPU costuma causar throttling, então muitos times não o definem.
- **readinessProbe:** o pod só recebe tráfego quando está pronto. **livenessProbe:** reinicia o pod travado. Não faça o liveness depender do banco.
- **topologySpread:** espalha réplicas entre zonas.
- **Graceful shutdown:** trate `SIGTERM` e termine as requisições em andamento antes de sair.

Comandos `kubectl` essenciais

```
az aks get-credentials -g rg-app-prd-brazilsouth -n aks-app-prd
kubectl get pods -n app -o wide
kubectl describe pod <nome> -n app      # eventos: por que não subiu
kubectl logs <nome> -n app --previous   # log do container que caiu
kubectl top pods -n app                 # consumo de CPU/memória
kubectl rollout status deploy/api -n app
kubectl rollout undo deploy/api -n app  # rollback
```

Em produção, use **Helm** ou **Kustomize** para empacotar manifestos e **GitOps** (Flux ou Argo CD) para que o cluster reflita o que está no Git.

4. SQL Server e Azure SQL

Para este cenário, use **Azure SQL Database** (PaaS): mesmo motor do SQL Server, com backup, patch e alta disponibilidade gerenciados. SQL Server em VM só se justifica quando você precisa de recursos de instância (SQL Agent complexo, CLR, cross-database) e, nesse caso, avalie antes a **Managed Instance**.

Opções no Azure

Opção	Quando usar	Quem cuida do SO e patches
Azure SQL Database	Apps novas, um banco por serviço	Azure
Azure SQL Managed Instance	Migração de SQL Server com recursos de instância	Azure
SQL Server em VM (IaaS)	Controle total ou software legado	Você

Modelos de compra e tiers

- **vCore** (recomendado): você escolhe núcleos e memória. **DTU** é um pacote mais simples e menos flexível.
- **General Purpose:** bom custo, armazenamento remoto. Atende a maioria das APIs.
- **Business Critical:** SSD local, latência menor, réplica de leitura inclusa. Para cargas sensíveis a latência.
- **Hyperscale:** até 128 TB, réplicas de leitura rápidas, escala de armazenamento independente.
- **Serverless:** pausa e escala automaticamente. Bom para dev, não para produção 8h–20h com SLA.
- Ponto de partida para 100–250 RPS: General Purpose ou Hyperscale com 4 a 8 vCores, **zone redundant**. Valide com teste de carga e ajuste.

Fundamentos de modelagem e consulta

- **Chave primária** em toda tabela; prefira `BIGINT IDENTITY` ou GUID sequencial (GUID aleatório fragmenta o índice clustered).
- **Índice clustered:** define a ordem física da tabela (um por tabela). **Nonclustered:** índices auxiliares para filtros e joins.
- **Índice de cobertura:** `INCLUDE` das colunas do `SELECT` para evitar *key lookup*.

- Normalize até a 3ª forma normal; desnormalize só com evidência de desempenho.
- Evite `SELECT *`, funções em colunas do `WHERE` (quebram o uso do índice) e consultas N+1 vindas do ORM.
- Use **queries parametrizadas** sempre: evitam SQL injection e reaproveitam planos.
- Transações curtas. Considere `READ_COMMITTED_SNAPSHOT` (padrão no Azure SQL) para leituras não bloquearem escritas.

```
CREATE TABLE dbo.Pedido (
  PedidoId   BIGINT IDENTITY PRIMARY KEY,
  ClienteId  BIGINT       NOT NULL,
  Status     TINYINT      NOT NULL,
  Total      DECIMAL(12,2) NOT NULL,
  CriadoEm   DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE INDEX IX_Pedido_Cliente_Data
  ON dbo.Pedido (ClienteId, CriadoEm DESC)
  INCLUDE (Status, Total);
```

Conexões e resiliência

- **Connection pooling** no app (padrão no ADO.NET, HikariCP no Java). Dimensione: pods × tamanho do pool não pode passar do limite de sessões e workers do tier.
- **Retry com backoff** para erros transitórios (failover, throttling): é obrigatório em nuvem.
- Autenticação via **Entra ID + Managed Identity**, sem usuário e senha SQL.
- Acesso só por **Private Endpoint**, com acesso público desabilitado.

Alta disponibilidade, backup e DR

- **Zone redundancy:** réplicas em 3 zonas, failover automático.
- **Backups automáticos** com Point-in-Time Restore (padrão de 7 dias, configurável até 35) e retenção de longo prazo (LTR) para compliance.
- **Failover Groups / Geo-replication:** cópia em outra região para desastre regional.
- Defina **RPO** (quanto dado posso perder) e **RTO** (quanto tempo posso ficar fora) com o negócio, e **teste o restore** periodicamente.

Tuning e monitoramento

- **Query Store:** histórico de consultas e planos; encontre as mais caras e regressões.
- **Automatic tuning:** cria índices sugeridos e corrige planos ruins.
- Monitore CPU, *DTU/vCore %*, *log IO*, *deadlocks* e conexões no Azure Monitor.

- Leitura pesada (relatórios): use a réplica de leitura (`ApplicationIntent=ReadOnly`) ou um cache como **Azure Cache for Redis**.
- Migrações de schema versionadas (Flyway, Liquibase, EF Migrations ou DbUp) no pipeline, sempre compatíveis com a versão anterior do app (*expand/contract*).

5. Kafka

Kafka é um log distribuído de eventos: produtores gravam mensagens em tópicos, e consumidores leem no seu próprio ritmo. Ele desacopla serviços, absorve picos e permite reprocessar o passado. No Azure, comece pelo **Event Hubs com endpoint Kafka**: seu código usa o client Kafka normal, sem operar brokers.

Conceitos essenciais

Conceito	O que é
Broker	Servidor Kafka que guarda os dados
Topic	Categoria de eventos (ex.: `pedidos.criados`)
Partition	Fatia ordenada de um tópico. É a unidade de paralelismo
Offset	Posição de uma mensagem na partição
Key	Define a partição. Mesma key = mesma partição = ordem garantida
Producer	Quem publica
Consumer group	Conjunto de consumidores que divide as partições: cada partição é lida por um só membro do grupo
Retention	Por quanto tempo as mensagens ficam guardadas
Replication factor	Quantas cópias de cada partição existem

Regras que pegam iniciantes

- **Ordem só dentro da partição.** Se a ordem por pedido importa, use `pedidoId` como key.
- **Paralelismo máximo = número de partições.** 6 partições com 10 consumidores deixam 4 ociosos.
- **At-least-once é o padrão realista:** mensagens podem chegar duplicadas. Faça o consumidor **idempotente** (ex.: tabela de IDs processados ou `MERGE` por chave).

- **Commit do offset depois de processar**, não antes, para não perder mensagens.
- **Dead Letter Topic (DLT):** mensagens que falham após N tentativas vão para um tópico separado, para não travar a fila.
- **Schema:** use Avro, Protobuf ou JSON Schema com um Schema Registry (no Azure, o do Event Hubs) e evolua de forma compatível.
- **Outbox pattern:** para gravar no banco e publicar o evento sem inconsistência, grave o evento numa tabela *outbox* na mesma transação e publique a partir dela.

Configurações-chave do produtor

```
acks=all                      # só confirma após replicar
enable.idempotence=true       # evita duplicata no envio
compression.type=lz4
linger.ms=5                   # agrupa mensagens em lotes
```

Opções no Azure

Opção	Operação	Quando escolher
Event Hubs (protocolo Kafka)	Totalmente gerenciado	Padrão para este cenário. Escala por Throughput/Processing Units
Confluent Cloud no Azure	Gerenciado pela Confluent	Precisa de recursos Kafka completos: ksqlDB, Connect gerenciado, compactação avançada
Strimzi no AKS	Você opera os brokers	Controle total, time com experiência em Kafka
Kafka em VMs	Você opera tudo	Raramente vale a pena hoje

Atenção: o Event Hubs implementa o protocolo Kafka, mas não é Kafka idêntico. Alguns recursos (como certas APIs administrativas ou transações, dependendo do tier) têm limitações. Confira a página de compatibilidade Kafka do Event Hubs antes de decidir.

Dimensionamento para este cenário

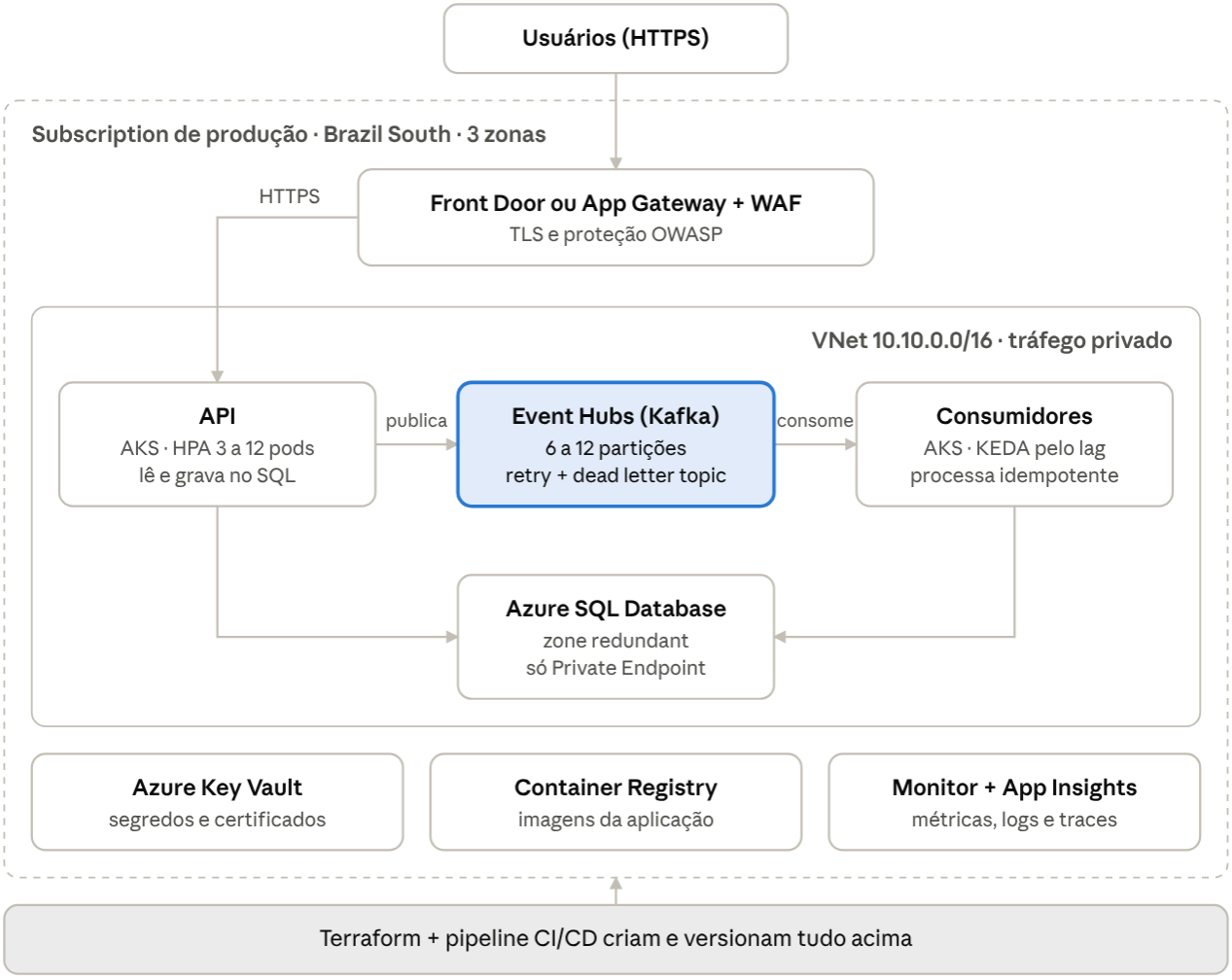
Mesmo que cada request gere 2 eventos, são cerca de 250 eventos/s no pico. Isso é pouco para Kafka: um namespace Event Hubs Standard com poucas unidades e tópicos com 6 a 12 partições atendem com folga e deixam margem para crescer. Escale consumidores com **KEDA** pelo *consumer lag* (quantas mensagens estão esperando).

6. Arquitetura de referência e dimensionamento

A API responde ao usuário lendo e gravando direto no SQL; o que pode ser feito depois (notificações, integrações, relatórios) vira evento no Kafka e é processado por consumidores. Tudo fica em uma região com 3 zonas, em rede privada, criado por Terraform.

Requests vão direto ao SQL; trabalho assíncrono passa pelo Kafka

Arquitetura de referência para 1M requests/dia, pico de até 250 RPS



arquitetura de referência · entrada, aplicação, eventos, dados

O usuário só enxerga o WAF; API, Kafka e banco ficam dentro da VNet, sem acesso público.

Dimensionamento inicial

Componente	Configuração inicial	Como escala
AKS node pool system	3 nós D4s_v5 (4 vCPU, 16 GB), um por zona	Fixo
AKS node pool user	3 a 6 nós D4s_v5 em 3 zonas	Cluster Autoscaler
API	3 a 12 pods, 0,5 vCPU e 512 MB cada	HPA a 65% de CPU
Consumidores	2 a 6 pods	KEDA pelo lag do tópico
Azure SQL	General Purpose ou Hyperscale, 4 a 8 vCores, zone redundant	Aumentar vCores ou adicionar réplica de leitura
Event Hubs	Standard, poucas unidades, tópicos com 6 a 12 partições	Auto-inflate de unidades
Entrada	Front Door ou Application Gateway v2 com WAF	Autoscaling nativo

Os números acima são ponto de partida, não verdade: confirme com o teste de carga do Lab 7 e ajuste. Uma API típica atende de 50 a 200 RPS por vCPU, então 250 RPS cabem em poucos pods; o gargalo provável é o banco.

Estimativa de custo

Monte a estimativa na Calculadora de Preços do Azure com a tabela acima, região Brazil South. Os maiores itens costumam ser, nesta ordem: nós do AKS, Azure SQL, ingestão de logs e WAF. Para reduzir:

- Reservations ou Savings Plan de 1 ano para os nós fixos e o banco
- Escalar o node pool user para o mínimo fora do horário 8h–20h
- Ambientes de dev e homologação menores e desligados à noite
- Amostragem de traces e retenção curta de logs de debug

7. Segurança, observabilidade, CI/CD e checklist de go-live

Produção confiável depende menos da tecnologia escolhida e mais de três hábitos: nada sem código, nada sem métrica, nada sem rollback.

Segurança

- Banco, Key Vault e Event Hubs **sem acesso público**, só por Private Endpoint.
- Entrada pública única com **WAF** (Application Gateway ou Front Door) e TLS.
- **Workload Identity** nos pods; zero senha em código, variável de ambiente ou repositório.
- **Network Policies** no AKS: cada serviço só fala com quem precisa.
- Scan de imagens no pipeline (Trivy ou Microsoft Defender for Containers) e **Azure Policy** para barrar containers privilegiados.
- **Microsoft Defender for Cloud** ativo; revise as recomendações semanalmente.
- Acesso administrativo com **PIM** (elevação temporária) e MFA.
- LGPD: dados pessoais mapeados, criptografados (TDE é padrão no Azure SQL) e com retenção definida.

Observabilidade

- **Os três sinais:** métricas (Azure Monitor / Managed Prometheus + Grafana), logs (Log Analytics) e traces (Application Insights com OpenTelemetry).
- **Golden signals por serviço:** latência (p50, p95, p99), taxa de erro, throughput (RPS) e saturação (CPU, memória, conexões do banco, lag do Kafka).
- **SLO:** ex.: 99,9% das requisições com sucesso e p95 abaixo de 300 ms no horário comercial. Alerte quando o *error budget* estiver sendo consumido rápido demais.
- Logs estruturados em JSON com `traceId` em toda linha.
- Cuidado com custo de log: Log Analytics cobra por GB ingerido. Defina níveis e amostragem.

CI/CD

1. **Pull request:** build, testes unitários, lint, `terraform plan` comentado no PR, scan de segurança.
2. **Merge na main:** build da imagem com tag do commit, push no ACR.
3. **Deploy em homologação:** automático, com testes de integração e smoke test.
4. **Deploy em produção:** aprovação manual, rolling update (ou canary), verificação automática de métricas.
5. **Rollback:** `kubectl rollout undo` ou revert no Git (GitOps). Precisa levar menos de 5 minutos.

Ferramentas comuns: GitHub Actions ou Azure DevOps Pipelines, com autenticação no Azure via **OIDC (federated credentials)**, sem segredo de service principal.

Checklist de go-live

- [ ] Toda infraestrutura criada por Terraform, com state remoto e pipeline
- [ ] AKS em 3 zonas, tier Standard, autoscaler e HPA configurados
- [ ] Requests, limits, probes e PDB em todos os Deployments
- [ ] Azure SQL zone redundant, Private Endpoint, restore testado
- [ ] Event Hubs/Kafka com DLT, consumidores idempotentes e alerta de lag
- [ ] WAF na entrada, sem acesso público a banco e cofre
- [ ] Dashboards e alertas dos golden signals, com plantão definido
- [ ] Teste de carga a 2x o pico (cerca de 250 RPS) por pelo menos 1 hora
- [ ] Teste de caos: derrubar um pod, um nó e forçar failover do banco
- [ ] Runbook de incidentes e de rollback escrito e ensaiado
- [ ] Budget e alertas de custo configurados

8. Trilha de estudos (12 semanas)

Com cerca de 8 a 10 horas por semana, você sai do zero e chega a montar a stack completa em homologação em 12 semanas. Cada semana termina com um laboratório da seção 9 e com as perguntas da seção 10.

Semana	Tema	Entrega da semana	Material de apoio
1	Linux, redes (IP, DNS, HTTP, TLS) e Git	Explicar o caminho de uma requisição do navegador ao servidor	Documentação MDN sobre HTTP
2	Fundamentos de Azure	Conta criada, budget configurado, Lab 1 feito	Trilha AZ-900 no Microsoft Learn
3	Rede e identidade no Azure	VNet, subnets, NSG e Managed Identity criados pelo portal e pela CLI	Trilha AZ-104 (módulos de rede e identidade)
4	Terraform básico	Lab 2: rede inteira em Terraform com state remoto	Tutoriais Azure em developer.hashicorp.com
5	Docker	API simples containerizada e publicada no ACR	Documentação Docker “Get started”

Semana	Tema	Entrega da semana	Material de apoio
6	Kubernetes local	Deployment, Service, Ingress e HPA rodando em kind ou minikube	kubernetes.io (Tutorials)
7	AKS com Terraform	Lab 3: AKS em 3 zonas com a API publicada	Documentação AKS no Microsoft Learn
8	SQL Server	Lab 4: schema, índices, análise de plano de execução	Microsoft Learn: Azure SQL fundamentals
9	Kafka	Lab 5: produtor e consumidor idempotente com DLT	Documentação Apache Kafka e Event Hubs for Kafka
10	Observabilidade e CI/CD	Lab 6: pipeline completo e dashboard dos golden signals	Documentação Application Insights e GitHub Actions
11	Teste de carga e resiliência	Lab 7: 250 RPS sustentados e teste de caos	Documentação k6 ou Azure Load Testing
12	Revisão e simulado	Apresentar a arquitetura a alguém e responder todas as perguntas da seção 10	Este guia

Certificações opcionais que validam a trilha: AZ-900 (fundamentos), AZ-104 (administrador), HashiCorp Terraform Associate e CKAD (desenvolvedor Kubernetes).

Dica de custo: use a conta gratuita ou créditos do Azure, crie tudo com Terraform e rode `terraform destroy` ao fim de cada sessão. Um AKS esquecido ligado no fim de semana custa caro.

9. Exercícios e laboratórios práticos

Os sete laboratórios constroem, juntos, a arquitetura da seção 6 em escala reduzida. Marque cada item ao concluir.

Lab 1: Explorando o Azure (semana 2)

- [ ] Criar um resource group pela CLI: `az group create -n rg-lab -l brazilsouth`
- [ ] Criar um Budget de R$ 100 com alerta em 80%

- [ ] Criar uma VM Linux pequena, conectar por SSH e depois apagá-la
- [ ] **Exercício:** explicar por escrito a diferença entre subscription, resource group e tenant

Lab 2: Rede em Terraform (semana 4)

- [ ] Criar o Storage Account do state e configurar o backend `azurerm`
- [ ] Escrever um módulo `network` com VNet `10.10.0.0/16` e subnets para AKS, banco e gateway
- [ ] Adicionar NSG que só libera 443 de entrada na subnet do gateway
- [ ] Rodar `plan`, `apply`, alterar um CIDR, ler o plano e explicar por que ele recria a subnet
- [ ] **Exercício:** usar `for_each` para criar as subnets a partir de um mapa

Lab 3: AKS com aplicação (semana 7)

- [ ] Criar ACR e AKS (3 zonas, autoscaler 2–4 nós) via Terraform, com permissão `AcrPull` para o cluster
- [ ] Publicar uma API simples com o Deployment, HPA e PDB da seção 3
- [ ] Expor por Ingress com HTTPS
- [ ] Apagar um pod e observar a recriação; fazer `kubectl drain` de um nó e ver o PDB agir
- [ ] **Exercício:** quebrar a readinessProbe de propósito e diagnosticar com `kubectl describe`

Lab 4: Azure SQL (semana 8)

- [ ] Criar Azure SQL com Private Endpoint e login via Entra ID
- [ ] Criar a tabela `Pedido` e inserir 1 milhão de linhas de teste
- [ ] Rodar uma consulta por cliente sem índice, ver o plano (*table scan*), criar o índice e comparar tempo e leituras (`SET STATISTICS IO ON`)
- [ ] Fazer um Point-in-Time Restore para um banco novo
- [ ] **Exercício:** a API acessa o banco com Workload Identity, sem senha

Lab 5: Kafka (semana 9)

- [ ] Criar namespace Event Hubs e tópico `pedidos.criados` com 6 partições
- [ ] Produtor com `acks=all` e idempotência, usando `pedidoId` como key
- [ ] Consumidor que grava no SQL de forma idempotente (reenviar a mesma mensagem não duplica)
- [ ] Enviar uma mensagem inválida e verificar que ela vai para a DLT

- [ ] **Exercício:** escalar consumidores com KEDA pelo lag e observar o limite de 6 consumidores ativos

Lab 6: CI/CD e observabilidade (semana 10)

- [ ] Pipeline com GitHub Actions: build, teste, scan, push no ACR, deploy em homologação
- [ ] Autenticação no Azure via OIDC
- [ ] Application Insights com OpenTelemetry e um trace ponta a ponta (API, banco, Kafka)
- [ ] Dashboard com latência p95, taxa de erro, RPS e lag; alerta para erro acima de 1%

Lab 7: Carga e caos (semana 11)

- [ ] Script k6 que sobe a 250 RPS e mantém por 30 minutos
- [ ] Registrar p95, erros, número de pods e nós, CPU do banco
- [ ] Durante a carga: apagar pods, drenar um nó, forçar failover do banco. Medir o impacto
- [ ] **Exercício final:** escrever um relatório de uma página com gargalos encontrados e custo mensal estimado

10. Perguntas que vão te fazer, com respostas

Cada público pergunta coisas diferentes: o entrevistador testa conceitos, o gerente quer risco, prazo e custo, e outros devs querem saber como usar no dia a dia. Treine respondendo em voz alta em até 1 minuto antes de ler a resposta.

Entrevista técnica

1. **Qual a diferença entre IaaS, PaaS e SaaS?** IaaS: você gerencia do SO para cima (VMs). PaaS: só app e dados (Azure SQL). SaaS: só usa (Microsoft 365). Prefiro PaaS para reduzir operação.
2. **O que é o state do Terraform e por que é crítico?** É o mapa entre o código e os recursos reais. Fica remoto (Storage Account com lock e versionamento), separado por ambiente, e nunca no Git porque contém segredos.
3. **O que acontece num `terraform plan` que mostra "forces replacement"?** O recurso será destruído e recriado, porque um atributo imutável mudou. Em banco ou cluster isso pode significar perda de dados ou downtime: paro e reavalio.
4. **Como lidar com drift?** Plan agendado no pipeline para detectar, e corrigir no código (ou `terraform import` quando o recurso foi criado fora). Mudanças manuais em produção são proibidas.

5. **Diferença entre Deployment, Service e Ingress?** Deployment mantém as réplicas dos pods; Service dá um endereço estável a elas; Ingress roteia HTTP de fora para os Services.
6. **Diferença entre readiness e liveness probe?** Readiness decide se o pod recebe tráfego; liveness decide se ele é reiniciado. Liveness não deve depender do banco, senão uma queda do banco reinicia todos os pods.
7. **Como o AKS escala?** HPA aumenta pods por métrica; quando faltam recursos, o Cluster Autoscaler adiciona nós. Para consumidores de fila, KEDA escala pelo lag.
8. **Para que servem requests e limits?** Requests reservam recurso e guiam o scheduler e o HPA. Limit de memória evita que um pod derrube o nó (o pod é morto por OOM).
9. **Clustered vs nonclustered index?** Clustered define a ordem física da tabela (um por tabela). Nonclustered é uma estrutura separada que aponta para as linhas; com `INCLUDE`, cobre a consulta e evita key lookup.
10. **Como você investiga uma query lenta?** Query Store para achar a query e o plano, plano de execução para ver scans e lookups, `STATISTICS IO` para leituras, e então índice ou reescrita.
11. **Como o Kafka garante ordem?** Só dentro de uma partição. Uso uma key de negócio (ex.: pedidoId) para que eventos da mesma entidade caiam na mesma partição.
12. **At-least-once, at-most-once ou exactly-once?** At-least-once é o realista: posso receber duplicatas, então o consumidor é idempotente. Exactly-once existe no Kafka, mas com custo e restrições, e não cobre efeitos externos como gravar num banco.
13. **Como gravar no banco e publicar um evento sem inconsistência?** Outbox pattern: grava o evento numa tabela na mesma transação e um processo separado publica no Kafka.
14. **Um consumer group com 10 consumidores e um tópico com 6 partições: o que acontece?** Só 6 consomem; 4 ficam ociosos. Para mais paralelismo, aumento as partições.

Gerente ou liderança

1. **Essa arquitetura aguenta 1 milhão de requests por dia?** Sim, com folga. Isso dá cerca de 23 RPS em média e até cerca de 120 no pico. Vamos dimensionar e testar para 250 RPS, e o autoscaling cobre crescimento.
2. **Quanto vai custar?** Depende do tier, mas os maiores itens são nós do AKS, banco e logs. Trago uma estimativa da calculadora do Azure e usamos reservas para reduzir 30% ou mais na carga estável. Teremos budget com alertas.
3. **Qual a disponibilidade esperada?** Com AKS e banco em 3 zonas, sobrevivemos à queda de um datacenter. Proponho SLO de 99,9% no horário comercial, que permite cerca de 22 minutos de indisponibilidade por mês no horário 8h–20h.

4. **E se a região inteira cair?** Precisamos definir RPO e RTO com o negócio. Multi-região ativa é cara; o comum é geo-replicação do banco e infraestrutura recriável por Terraform em outra região.
5. **Por que Kafka? Não é complexidade demais?** Kafka desacopla serviços e absorve picos sem derrubar o banco. Usando Event Hubs gerenciado, não operamos brokers. Se não houver processamento assíncrono real, podemos adiar.
6. **Quanto tempo para colocar em produção?** Com o time familiarizado, a base de plataforma leva de 4 a 8 semanas incluindo testes de carga e checklist de go-live; depois, cada serviço novo é bem mais rápido porque reutiliza os módulos.
7. **Quais são os maiores riscos?** Falta de experiência do time em Kubernetes, custo sem controle e mudanças manuais fora do Terraform. Mitigação: treinamento, budgets e pipeline obrigatório.
8. **Estamos em conformidade com a LGPD?** Dados ficam em Brazil South, criptografados em repouso e em trânsito, com acesso controlado por Entra ID e auditoria. Falta mapear dados pessoais e retenção com o jurídico.

Outros desenvolvedores

1. **Como faço deploy do meu serviço?** Abra PR; após merge, o pipeline gera a imagem e faz deploy em homologação. Produção tem aprovação manual. Você só mantém o Helm chart ou os manifestos do serviço.
2. **Como pego a connection string do banco?** Não existe senha: o pod usa Workload Identity e o driver autentica no Entra ID. Configurações não secretas ficam em ConfigMap; segredos de terceiros, no Key Vault.
3. **Meu pod está em CrashLoopBackOff. E agora?** `kubectl describe pod` para ver eventos e `kubectl logs --previous` para o log do container que caiu. Causas comuns: configuração faltando, OOM ou liveness agressiva.
4. **Como vejo logs e traces?** Application Insights: busque pelo `traceId`. Logs estruturados vão para o Log Analytics.
5. **Posso criar um recurso pelo portal rapidinho?** Em dev, para aprender, sim. Em homologação e produção, só via Terraform e PR, senão geramos drift.
6. **Como crio um tópico novo?** Via Terraform (módulo `eventhubs`), com nome no padrão `dominio.evento`, partições definidas pelo paralelismo esperado e schema registrado.
7. **Como faço migração de schema sem downtime?** Expand/contract: primeiro adiciono colunas compatíveis, publico o app que usa as duas formas, migro os dados e só depois removo o antigo.
8. **Meu consumidor processou a mesma mensagem duas vezes. Bug do Kafka?** Não: at-least-once permite duplicatas. O consumidor precisa ser idempotente, por exemplo checando o ID do evento antes de aplicar.